

Module 7 — React: Displaying Data Lists & CSS

Time: 1 hour · Covers Practical 6 · CO3

Practical 6: Using components to display a set of data together on screen. Also create a CSS for the same.

Part A — Rendering a list with `.map()` (concept, ~20 min)

To show many items, you turn an **array of data** into an **array of JSX** with JavaScript's `.map()`. React renders each element.

```
const fruits = ["Apple", "Banana", "Cherry"];

function FruitList() {
  return (
    <ul>
      {fruits.map((fruit) => (
        <li key={fruit}>{fruit}</li>
      ))}
    </ul>
  );
}
```

The `key` prop — required!

Each element in a mapped list needs a unique `key`. React uses it to track which items changed, so it can update the screen efficiently. Use a stable unique id (like a database `_id`), **not** the array index when the list can reorder.

```
{users.map((u) => <li key={u.id}>{u.name}</li>)}
```

Part B — Passing data with props (~15 min)

Props ("properties") are how a parent component passes data *down* to a child. They are read-only inside the child.

```
// Child: receives `user` via props and displays it.
function UserCard({ user }) {
  return (
    <div className="user-card">
      <h3>{user.name}</h3>
      <p>{user.email}</p>
    </div>
  );
}

// Parent: passes each user as a prop.
function UserList({ users }) {
```

```

return (
  <div>
    {users.map((u) => (
      <UserCard key={u.id} user={u} />
    ))}
  </div>
);
}

```

`{ user }` in the parameter list is **destructuring** — it pulls the `user` field out of the props object.

Read the code in this order: [src/data.js](#) (the data) → [src/components/UserCard.js](#) (one item) → [src/components/UserList.js](#) (the list) → [src/App.js](#) .

Part C — Styling with CSS (~15 min)

Two common ways to style in React:

1. External CSS file + `className`

```

import './App.css';
<div className="user-card">...</div>

```

```

/* App.css */
.user-card {
  border: 1px solid #ddd;
  border-radius: 10px;
  padding: 1rem;
}

```

2. Inline styles (a JS object, camelCased keys)

```

// note: double braces — outer {} = JS expression, inner {} = the style object
<span style={{ color: "white", backgroundColor: "green", padding: "2px 6px" }}>
  active
</span>

```

Use external CSS for most styling; inline styles for small, dynamic tweaks.

Conditional styling

Choose a class based on data:

```

<span className={user.active ? "badge badge-on" : "badge badge-off"}>
  {user.active ? "Active" : "Inactive"}
</span>

```

The full grid of styled user cards is in [src/App.css](#) — it uses CSS **grid** to lay the cards out responsively.

Part D — Run it (~10 min)

```
cd 07-react-data-css
npm install
npm start          # opens http://localhost:3000
```

You should see a responsive grid of user cards, each with a colored active/inactive badge.

Summary

- Turn a data array into UI with `.map()` ; give each item a unique `key` .
- **Props** pass data from parent to child (read-only in the child).
- Style with an external **CSS file + `className`** , or **inline style objects** for dynamic bits.
- Pick a class **conditionally** to reflect data (active/inactive).

Now do [practice.md](#) .